
Práctica 3 : Monitorización, Automatización y "Profiling"

Índice

1	Introducción	4
2	Monitores para Hardware	4
2.1	Mensajes del kernel: dmesg	5
3	Monitores para Software	5
3.1	Subsistema de archivos	5
3.2	Monitorizando un servicio o ejecución de un programa: strace	5
4	Monitores generales	5
4.1	Munin	6
4.2	Nagios y Naemon	6
4.3	Ganglia	6
4.4	Grafana	6
4.5	Netdata	7
4.6	Elastic (pila ELK)	7
4.7	ZABBIX	7
5	Automatización	8
5.1	cron y systemd	8
5.2	Scripts	9
5.2.1	Shell y comandos del sistema: grep, find, awk y sed	9
5.2.2	Python y PHP	9
5.3	A nivel de Plataforma: Ansible	10
6	Profiling	10
6.1	Scripts	10
6.2	SQL	11

OBJETIVOS MÍNIMOS

1. Conocer y saber usar las herramientas que permitan obtener datos sobre el sistema a nivel hardware y software (SO y servicios).
2. Saber interpretar los resultados proporcionados por las aplicaciones de monitorización.
3. Conocer los archivos que proporcionan información del sistema.
4. Tener conocimiento básico sobre automatización y orquestación
5. Ser capaz de configurar y utilizar un monitor de sistema

Contenido de las lecciones

1. Monitorización del RAID1, Monitores y Automatización
2. Zabbix

Competencias que se trabajarán

Competencias Básicas

1. CB2. Que los estudiantes sepan aplicar sus conocimientos a su trabajo o vocación de una forma profesional y posean las competencias que suelen demostrarse por medio de la elaboración y defensa de argumentos y la resolución de problemas dentro de su área de estudio.

Específicas de la Asignatura

1. R1. Capacidad para diseñar, desarrollar, seleccionar y evaluar aplicaciones y sistemas informáticos, asegurando su fiabilidad, seguridad y calidad, conforme a principios éticos y a la legislación y normativa vigente.
2. R2. Capacidad para planificar, concebir, desplegar y dirigir proyectos, servicios y sistemas informáticos en todos los ámbitos, liderando su puesta en marcha y su mejora continua y valorando su impacto económico y social.
3. R5. Conocimiento, administración y mantenimiento de sistemas, servicios y aplicaciones informáticas.

Competencias Específicas del Título:

1. E4. Capacidad para definir, evaluar y seleccionar plataformas hardware y software para el desarrollo y la ejecución de sistemas, servicios y aplicaciones informáticas.

Competencias Transversales o Generales:

1. T2. Capacidad de organización y planificación así como capacidad de gestión de la Información.

1. Introducción

Esta práctica consiste en la utilización de herramientas de monitorización del sistema para visualizar cómo se comporta el sistema ante ciertas actividades que los usuarios u otros servicios generan.

Las herramientas de monitorización presentan medidas del sistema permitiendo generar informes e históricos que puedan ser de utilidad para un análisis a posteriori (off-line) o para tomar decisiones sobre la marcha (on-line).

En última instancia, el objetivo de esta práctica es monitorizar varios sistemas que tienen servicios instalados siendo consciente de cómo se realiza el proceso tanto a bajo nivel como usando herramientas que nos permiten trabajar a alto nivel.

A raíz de la solución basada en Grafana y la inclusión de IA en los stacks de monitorización, se está extendiendo el uso del concepto de *observabilidad* que difiere en matices filosóficos de la monitorización añadiendo el componente de predecir y añadir acciones proactivas en base a las métricas y su análisis. Según Amazon “ *El monitoreo recopila datos sobre los componentes individuales y la observabilidad analiza el sistema distribuido como un conjunto.*”¹. En el caso de IBM, podemos encontrar una referencia [1] con un contenido bastante completo y correcto, no deje de consultarlo para aprender sobre el término y continuar con la disciplina fundamental del *Site Reliability Engineer* que puede ser uno de sus múltiples itinerarios profesionales. Le dejamos unas referencias sobre el tema por si desea ir profundizando: <https://sre.google/books/>.

2. Monitores para Hardware

Además del estado del software del servidor, también existen programas que nos permiten ver el estado del hardware de nuestra máquina. En primer lugar, muchas BIOS (Basic Input Output System) (ya en extinción debido a la aparición de los Universal Extended Firmware, UEFI) nos permiten acceder a cierta información sobre el estado del HW, sin embargo, para no tener que reiniciar, podemos utilizar otras herramientas. Concretamente, para Linux está: `hddtemp` para la temperatura del HD tenemos y el proyecto `lm-sensors`: <https://github.com/lm-sensors/lm-sensors> (con su correspondiente GUI: `xsensors`).

Al estar trabajando con máquinas virtuales, la ejecución y obtención de resultados de estos monitores no aporta nada ya que el hardware que monitorizan es virtual.

No obstante, debemos ser conscientes de que hay ciertos comandos que ya hemos visto que nos permiten listar el hardware disponible: `lspci`, `lsusb`, `lshw` nos muestran los dispositivos conectados a los buses e información general sobre el HW del equipo. Pruebe a ejecutarlos y ver qué muestran.

¹<https://aws.amazon.com/es/compare/the-difference-between-monitoring-and-observability/>

2.1. Mensajes del kernel: dmesg

El kernel de Linux permite conocer qué actividad ha ocurrido gracias a los mensajes que proporciona el kernel. Esto es especialmente útil para detectar problemas con el HW o periféricos.

3. Monitores para Software

3.1. Subsistema de archivos

En linux (UNIX) todo se manipula a través de archivos de una manera cómoda y transparente. Existe un directorio especial: `/proc` (visto en clase de teoría) y `/var` (algo se ha comentado también al respecto) que nos pueden dar información tanto del hardware como del software.

En estos directorios se encuentran archivos fundamentales en la monitorización del comportamiento del sistema, de las aplicaciones y de los usuarios. Cabe destacar el subdirectorio `/var/log` que contiene los archivos en los que se van volcando los logs de los servicios y algunas aplicaciones.

Hay que tener ciertas precauciones ya que, una mala gestión de los archivos de bitácora puede resultar en un sistema caído. Para evitar que los logs crezcan indefinidamente, se realiza una rotación de estos archivos (`logrotate`).

Gracias a estos archivos podemos identificar errores y problemas, además esta tarea puede ser automatizada. Como ejemplo retomaremos los RAID1 en Ubuntu server de la primera práctica.

Usted debe conocer cómo identificar, monitorizar y reconstruir los discos RAID que tiene configurados en sus máquinas virtuales. Puede encontrar información imprescindible en las páginas del manual para `mdadm` y el archivo `/proc/mdstat` además de en [2, 3].

3.2. Monitorizando un servicio o ejecución de un programa: `strace`

Hay un conjunto de programas que permiten hacer una traza de las llamadas al sistema realizadas por un programa (servicio) en ejecución, p.ej. `strace` (system call tracer). Este tipo de programas pueden ser útiles de cara a detectar problemas que no se muestran en los archivos de “log”.

En <http://chadfowler.com/2014/01/26/the-magic-of-strace.html> tiene ejemplos de uso y podrá apreciar la utilidad de este tipo de programas.

4. Monitores generales

Además de los comandos integrados vistos en teoría y los que han usado en prácticas, existen otros programas muy populares que permiten monitorizar el sistema.

Hay algunos de entorno corporativo de grandes empresas como NetApp (<http://www.netapp.com/es/products/management-software/>), aunque los que se verán en las siguientes subsecciones también son usados por grandes instituciones y empresas.

4.1. **Munin**

Munin (significa memoria) está disponible en <http://munin-monitoring.org/>. Para su instalación (en CentOS) puede hacerlo compilando el código fuente (como ha podido hacer con `lm_sensors`) alojado en la página o a través del paquete disponible en el repositorio EPEL (<http://fedoraproject.org/wiki/EPEL/es>). “¿Cómo puedo utilizar estos paquetes adicionales?” es el título de la subsección dentro de la página donde se explica cómo activar el repositorio que contiene los paquetes. Tan solo debe instalar un `.rpm` y podrá usar `yum` para instalar `munin`. Para Ubuntu, está disponible sin tener que añadir ningún repositorio adicional. En <http://demo.munin-monitoring.org/> puede ver una demo del sistema de monitorización en funcionamiento.

4.2. **Nagios y Naemon**

Es otro software muy usado para monitorizar sistemas. Recientemente ha pasado por una transformación de proyecto de la comunidad a proyecto empresarial. Debido a esto, ha aparecido Naemon <http://naemon.org> que acaba de hacer una nueva *release*. Para ver una demo, podemos identificarnos a través de la demo que hace uso de la popular interfaz Thruk (<https://demo.thruk.org/demo/thruk/cgi-bin/login.cgi?demo/thruk>).

4.3. **Ganglia**

Es un proyecto alojado en <http://ganglia.sourceforge.net/> que monitoriza sistemas de cómputo distribuidos (normalmente para altas prestaciones) y permite una gran escalabilidad. Como puede verse en su página, importantes instituciones y compañías lo usan (p.ej. UC Berkely, MIT, Twitter, ...).

4.4. **Grafana**

Este software se autodefine como la pila de observabilidad (cualquier cosa que desee monitorizar) [4]. Para construir su pila tiene distintos componentes que se estructuran en complementos (plugins), bitácoras (logs), paneles (dashboards) y alertas (alerts). Dentro de esta visión amplia de la monitorización, incluyen un sistema de traza que se solapa con el concepto de *profiling*.

Es una solución de código abierto con posibilidad de contratar su servicio alojado en la nube y de manera autogestionada para empresas con mayor presupuesto.

Dentro de las empresas que han adoptado esta tecnología encontramos algunas muy populares y muchos casos de éxito de implantación tienen un artículo asociado explicando cómo

la han adoptado, se le anima a revisar algunos de éstos que suelen incluir algún vídeo explicativo (como, por ejemplo, el de una famosa empresa de citas: [https://grafana.com/blog/2019/03/26/tinder-grafana-a-love-story-in-metrics-and-monitoring/?plcm=](https://grafana.com/blog/2019/03/26/tinder-grafana-a-love-story-in-metrics-and-monitoring/?plcm=footer) footer).

Como suele ser habitual, tenemos la posibilidad de jugar con una demo disponible en: <https://play.grafana.org/d/000000012/grafana-play-home?orgId=1>.

4.5. Netdata

Otra empresa que ha ido desarrollando un producto un poco más enfocado a nicho ha sido Netdata. Su especialización ha sido la minimización del impacto de la solución de monitorización pudiendo observar los datos en tiempo real.

Recientemente han incorporado otros aspectos (ya presentes en otras soluciones de monitorización) como la inclusión de aprendizaje automático para generar alertas. Además, conscientes de la popularización de Grafana y viendo que es una solución que abarca más aspectos, han desarrollado una integración para esta otra solución de monitorización.

Es un proyecto opensource y miembro de la Linux Foundation. Pueden probar el producto en https://london.my-netdata.io/#menu_system_submenu_cpu;after=-480;before=0;theme=slate;help=true;utc=Europe/London.

4.6. Elastic (pila ELK)

Aunque no empezó su desarrollo como una herramienta de monitorización sino como una de búsqueda de información, el ecosistema ha crecido y se ha definido la pila ELK: Elastic + Logstash + Kibana. Gracias a estos tres componentes, se puede almacenar información (logstash) mediante lo que denominan *beats*, indexarla y hacer búsquedas eficientes (Elastic) y visualizarla en paneles (Kibana).

Existen más plugins y es una solución bastante popular por sus integraciones, no obstante, es software privativo.

4.7. ZABBIX

Es otro programa que permite monitorizar el sistema también de código abierto. Su instalación es relativamente sencilla ya que solo hay que añadir los repositorios (visto en la práctica anterior) e instalarlo con el gestor de paquetes.

Este es el sistema de monitorización en el que nos centraremos en las lecciones. En [5] encontrará toda la información que necesitará para el seguimiento de las lecciones.

El objetivo es que usted sea capaz de instalar este sistema de monitorización y configurar éste para que se monitorice a sí mismo así como a Rocky (a través del agente) además de conocer cómo interactuar con el sistema de monitorización usando la API que proporcionar y conocer en qué consiste el protocolo SNMP.

Le proponemos el siguiente ejercicio que debe realizar de manera obligatoria:

Ejercicio 1:

Realice una instalación de Zabbix 5.0 en su servidor con Ubuntu Server 20.04 y configure para que se monitoree a él mismo y para que monitorice a la máquina con Rocky Linux. Puede configurar varios parámetros para monitorizar, uso de CPU, memoria, etc. pero debe configurar de manera obligatoria, como mínimo, la monitorización de los servicios SSH y HTTP.

Es recomendable, para reforzar el aprendizaje, que documente el proceso de instalación y configuración indicando las referencias que ha utilizado así como los problemas que ha encontrado.

5. Automatización

5.1. cron y systemd

Tradicionalmente, el servicio cron ha permitido ejecutar cada cierto intervalo de tiempo una tarea concreta. Esto es muy útil de cara a recopilar información o monitorizar el sistema realizando una tarea concreta y lanzar alertas como, por ejemplo, enviar un correo electrónico cuando la carga esté por encima de un valor determinado (aunque algunos comandos de monitorización permitan hacer esa tarea directamente).

Con la introducción de systemd, cron puede seguir siendo usado pero su funcionamiento está basado en los *timers* que activan *services*. La documentación más recomendable sobre la gestión de systemd y los servicios está en los manuales de Red Hat, concretamente en https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/system_administrators_guide/sect-managing_services_with_systemd-unit_files, aunque en https://people.redhat.com/pladd/systemd_NYRHUG_2016-03.pdf tiene un resumen con más enlaces a documentación.

Como ejemplo, podemos tomar este script escrito en Python que nos permite monitorizar si algún dispositivo de nuestro RAID ha fallado (puede ver más ejemplos en http://echorand.me/site/notes/articles/python_linux/article.html):

```
import re
f=open( '/proc/mdstat ' )
for line in f:
    b=re.findall( ' \[ [U]* [_]+ [U]* \] ', line )
    if (b!= []):
        print( "—ERROR_en_RAID—" )
print( "—OK_Script—" )
```

Podemos automatizar la ejecución de este script en systemd definiendo un *timer* dentro del directorio `/etc/systemd/system/` que se encarga de gestionar un servicio. Por tanto, hemos de crear dos archivos: `mon RAID.timer` y `mon RAID.service`

```
[Unit]
Description=Monitor RAID status
[Timer]
OnCalendar=minutely
```

```
[Install]
WantedBy=timers.target}

[Unit]
Description=Monitor RAID status
[Service]
Type=simple
ExecStart=/usr/bin/python3 /home/alberto/mon-raid.py
```

Para que pueda funcionar hemos de activar y habilitar el *timer* (de manera análoga a lo que hicimos con los servicios tras instalarlos en la Práctica 2). Una vez hecho esto, podremos monitorizar su ejecución gracias al comando `journalctl`, p.ej. `journalctl -u mon-raid --since="yesterday"`.

5.2. Scripts

5.2.1. Shell y comandos del sistema: grep, find, awk y sed

Como ya han estudiado en Sistemas Operativos, el shell, en sus diferentes versiones (z, bourne, bash, c, ...) permite programar permitiendo hacer tareas de manera automática. Además de lo que ya saben, es importante mencionar tres comandos muy útiles de cara a automatizar tareas: `grep`, `find`, `awk` y `sed`, puede consultar algunos ejemplos en [6, 7]. Por ejemplo con `sed` puede buscar una cadena en un archivo y reemplazarla por otra, así podría dar acceso por contraseña durante unos instantes al servicio `ssh` para que los usuarios puedan copiar su llave pública.

Con `awk` puede programar la manipulación del texto así como generar salidas más completas a partir de la información en un archivo.

Con `grep` puede realizar filtrado de cadenas, útil cuando un archivo, listado, etc tiene unas dimensiones grandes, p.ej. `ps -Af | grep firefox` nos mostraría la información del proceso `firefox` (descartando el resto de información) Por último, con `find`, aunque probablemente ya lo haya usado en SOs, puede buscar archivos y, una vez encontrados, realizar acciones sobre ellos, p.ej. `find /home/alberto/docs -name '*pdf' -exec cp {} ~/PDFs \;` copiará todos los archivos cuyo nombre termine en `pdf` y los copia en la carpeta `/home/alberto/PDFs`

5.2.2. Python y PHP

Uno de los lenguajes más populares a día de hoy para programar servidores, concretamente páginas web dinámicas, es PHP. Existen numerosos y extendidos CMS (Content Management System) escritos en PHP, p.ej. Wordpress, Joomla, CakePHP, etc. Para poder programar sin tener que partir de cero existen varios frameworks como Symfony y Zend entre otros.

Python es un lenguaje de scripting con una creciente popularidad y se presenta como una gran alternativa a PHP (el framework `django` se basa en Python). También está ganando usuarios en el mundo de la investigación presentándose como una alternativa a Matlab (sin tener en cuenta Octave).

De cara a la asignatura, debido a su facilidad para manipular cadenas de texto mediante bibliotecas, nos puede permitir escribir tareas automatizadas que manipulen archivos de configuración. Como ejemplos muy sencillos que no requieren excesivo conocimiento del lenguaje tenemos los proporcionados en [8]. En [9] también hay otros scripts que muestran posibles casos prácticos además de todas las *recetas* disponibles en <https://github.com/ActiveState/code/tree/master/recipes/Python>. Como último ejemplo, es oportuno citar la biblioteca para hacer uso de la API de Zabbix disponible en [10].

5.3. A nivel de Plataforma: **Ansible**

Además de todo lo visto, también existen interfaces que permiten programar scripts y visualizar su ejecución de una manera cómoda y visual. Por falta de tiempo, no los cubriremos en la asignatura con detalle, no obstante, sí que veremos en una lección el funcionamiento básico de Ansible y se le anima a visitar las webs de los proyectos y ver algún vídeo relacionado.

P.ej. Puppetlabs <http://puppetlabs.com/> Ansible <http://www.ansible.com/home> Run-deck <http://rundeck.org/screencasts.html>

Ejercicio 2: Usted deberá saber cómo instalar y configurar Ansible para poder hacer un ping a las máquinas virtuales de los servidores y ejecutar un comando básico (p.ej. el script de monitorización del RAID1). También debe ser consciente de la posibilidad de escribir acciones más complejas mediante playbooks escritos con YAML como, por ejemplo, asegurarse de que tenemos la última versión instalada de httpd y que está en ejecución.

Aunque posee la documentación oficial de Ansible, es recomendable leer la entrada de “Ansible Basics” dentro del libro disponible en la documentación de Rocky Linux denominado “Learning Ansible”.

6. **Profiling**

6.1. Scripts

Para estudiar el comportamiento de los scripts, independientemente el lenguaje que usemos, podemos usar varios profilerers, p.ej. en Bash podemos usar la opción `set -x` y modificar la variable local PS4 para que muestre el tiempo http://www.tldp.org/LDP/Bash-Beginners-Guide/html/sect_03_02.html. En el caso de PHP tenemos el proyecto desarrollado inicialmente por Facebook:

XHProf: <http://pecl.php.net/package/xhprof> <https://github.com/facebook/xhprof>

Además de la documentación oficial, en la página de un prestigioso sistema de enseñanza virtual hay unos tutoriales interesantes sobre cómo realizar el profiling: http://docs.moodle.org/dev/Profiling_PHP.

En el caso de Python se puede utilizar el módulo cProfile. Para más información consulte en <https://docs.python.org/3/library/profile.html>.

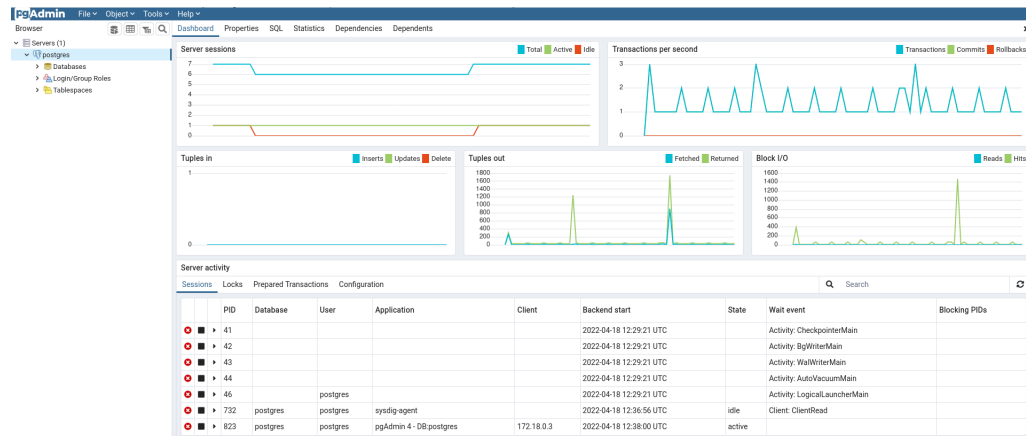


Figura 6.1: Figura de ejemplo de monitorización de la BD con pgadmin.

6.2. SQL

El mismo MySQL (y MariaDB) que instalamos en la práctica anterior tiene un “profiler” para analizar cuánto tardan las consultas.

Puede ver la documentación en:

<http://dev.mysql.com/doc/refman/5.0/en/show-profiles.html> <http://dev.mysql.com/doc/refman/5.0/en/show-profile.html>

Además, es especialmente útil la herramienta MySQLWorkBench y otros comandos disponibles como `mysqloptimize` que optimizan la ejecución.

Pruebe a obtener un profile de una tabla cogiendo todos los atributos (`SELECT * ...`) y a escoger únicamente uno o dos (`SELECT ID, AGE ...`) y compare el tiempo que requiere consultar información extra que no necesitamos.

Por otra parte, para PostgreSQL también disponemos de pgAdmin (<https://www.pgadmin.org/>) que nos permite monitorizar cómodamente nuestra base de datos con independencia del SO en el que la estemos ejecutando.

Referencias

- [1] IBM, “Observabilidad.” <https://www.ibm.com/es-es/topics/observability>, 2024. [Online; consultada 31-Octubre-2024].
- [2] David Greaves *et al.*, “Detecting, querying and testing.” https://raid.wiki.kernel.org/index.php/Detecting,_querying_and_testing, 2006–2011. [Online; consultada 10-Noviembre-2021].
- [3] David Greaves *et al.*, “mdstat.” <https://raid.wiki.kernel.org/index.php/Mdstat>, 2007–2013. [Online; consultada 10-Noviembre-2021].
- [4] Grafana Labs, “Granafa.” <https://grafana.com/>, 2022. [Online; consultada 31-Octubre-2022].

-
- [5] Zabbix SIA, “Manual de zabbix.” <https://www.zabbix.com/documentation/5.0/manual>, 2020. [Online; consultada 14-September-2020].
 - [6] T. G. Stuff, “Ejemplos de sed.” <http://www.thegeekstuff.com/2009/10/unix-sed-tutorial-advanced-sed-substitution-examples/>, 2017. [Online; consultada 10-Noviembre-2021].
 - [7] T. G. Stuff, “Ejemplos de awk.” <http://www.thegeekstuff.com/2010/01/awk-introduction-tutorial-7-awk-print-examples/>, 2017. [Online; consultada 10-Noviembre-2021].
 - [8] A. Saha, “Linux system mining with python.” http://echorand.me/site/notes/articles/python_linux/article.html, 2013. [Online; consultada 10-Noviembre-2021].
 - [9] J. K. (IBM), “Python for system administrators.” <https://www.ibm.com/developerworks/aix/library/au-python/index.html>, 2013. [Online; consultada 10-Noviembre-2021].
 - [10] L. Cyca, “Pyzabbix.” <https://github.com/lukecyca/pyzabbix>, 2017. [Online; consultada 10-Noviembre-2021].